

```
In [62]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
```

```
In [63]: df=pd.read_csv(r"C:\Users\Lenovo\Documents\Notes\APRIL\4th - REGRESSION PROJECT\RES
```

```
In [64]: df.head()
```

Out[64]:

	Unnamed: 0	Date	AveragePrice	Total Volume	4046	4225	4770	Total Bags	Small Bags
0	0	2015-12-27	1.33	64236.62	1036.74	54454.85	48.16	8696.87	8603.62
1	1	2015-12-20	1.35	54876.98	674.28	44638.81	58.33	9505.56	9408.07
2	2	2015-12-13	0.93	118220.22	794.70	109149.67	130.50	8145.35	8042.21
3	3	2015-12-06	1.08	78992.15	1132.00	71976.41	72.58	5811.16	5677.40
4	4	2015-11-29	1.28	51039.60	941.48	43838.39	75.78	6183.95	5986.26

```
In [65]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18249 entries, 0 to 18248
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Unnamed: 0      18249 non-null  int64
1   Date            18249 non-null  object
2   AveragePrice    18249 non-null  float64
3   Total Volume    18249 non-null  float64
4   4046            18249 non-null  float64
5   4225            18249 non-null  float64
6   4770            18249 non-null  float64
7   Total Bags      18249 non-null  float64
8   Small Bags      18249 non-null  float64
9   Large Bags      18249 non-null  float64
10  XLarge Bags     18249 non-null  float64
11  type            18249 non-null  object
12  year            18249 non-null  int64
13  region          18249 non-null  object
dtypes: float64(9), int64(2), object(3)
memory usage: 1.9+ MB
```

```
In [66]: df.isnull().sum()
```

```
Out[66]: Unnamed: 0      0
         Date          0
         AveragePrice  0
         Total Volume  0
         4046          0
         4225          0
         4770          0
         Total Bags    0
         Small Bags    0
         Large Bags    0
         XLarge Bags   0
         type          0
         year          0
         region        0
         dtype: int64
```

```
In [67]: df = df.drop(['Unnamed: 0', '4046', '4225', '4770', 'Date'], axis=1)
```

```
In [68]: df.head()
```

```
Out[68]:
```

	AveragePrice	Total Volume	Total Bags	Small Bags	Large Bags	XLarge Bags	type	year	region
0	1.33	64236.62	8696.87	8603.62	93.25	0.0	conventional	2015	Albany
1	1.35	54876.98	9505.56	9408.07	97.49	0.0	conventional	2015	Albany
2	0.93	118220.22	8145.35	8042.21	103.14	0.0	conventional	2015	Albany
3	1.08	78992.15	5811.16	5677.40	133.76	0.0	conventional	2015	Albany
4	1.28	51039.60	6183.95	5986.26	197.69	0.0	conventional	2015	Albany

```
In [69]: def get_avarage(df, column):
         """
         Description: This function to return the average value of the column

         Arguments:
             df: the DataFrame.
             column: the selected column.
         Returns:
             column's average
         """
         return sum(df[column])/len(df)
```

```
In [70]: def get_avarage_between_two_columns(df, column1, column2):
         """
         Description: This function calculate the average between two columns in the dat

         Arguments:
             df: the DataFrame.
             column1: the first column.
             column2: the sccond column.
```

```

Returns:
    Sorted data for relation between column1 and column2
    """

List=list(df[column1].unique())
average=[]

for i in List:
    x=df[df[column1]==i]
    column1_average= get_avarage(x,column2)
    average.append(column1_average)

df_column1_column2=pd.DataFrame({'column1':List,'column2':average})
column1_column2_sorted_index=df_column1_column2.column2.sort_values(ascending=False)
column1_column2_sorted_data=df_column1_column2.reindex(column1_column2_sorted_index)

return column1_column2_sorted_data

```

```

In [71]: def plot(data,xlabel,ylabel):
    """
    Description: This function to draw a barplot

    Arguments:
        data: the DataFrame.
        xlabel: the label of the first column.
        ylabel: the label of the second column.

    Returns:
        None
    """

    plt.figure(figsize=(15,5))
    ax=sns.barplot(x=data.column1,y=data.column2,palette='rocket')
    plt.xticks(rotation=90)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.title(('Avarage '+ylabel+' of Avocado According to '+xlabel));

```

```

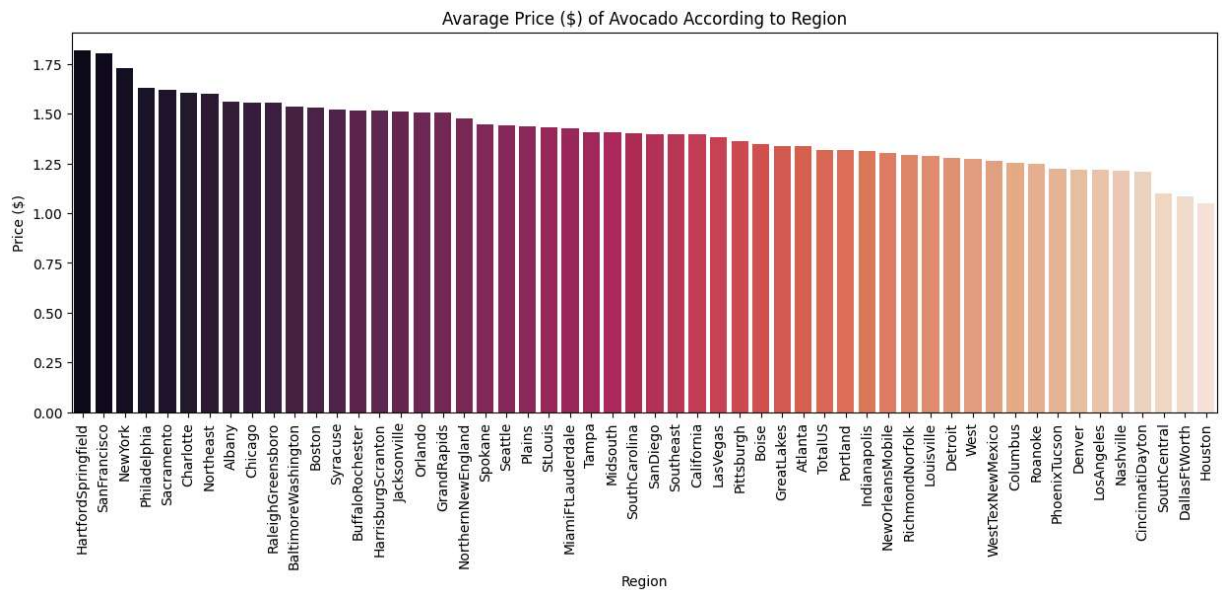
In [72]: data1 = get_avarge_between_two_columns(df,'region','AveragePrice')
plot(data1,'Region','Price ($)')

```

C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12444\640296719.py:14: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax=sns.barplot(x=data.column1,y=data.column2,palette='rocket')
```



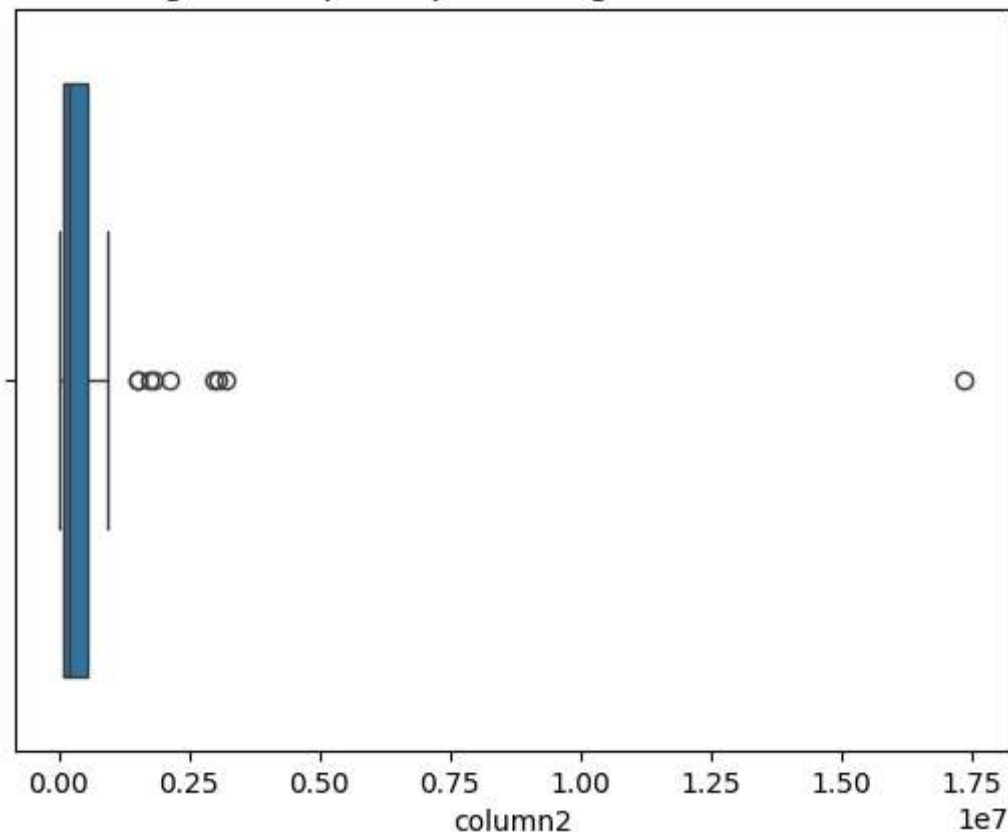
```
In [73]: print(data1['column1'].iloc[-1], " is the region producing avocado with the lowest p
```

Houston is the region producing avocado with the lowest price

```
In [74]: data2 = get_avarge_between_two_columns(df, 'region', 'Total Volume')
sns.boxplot(x=data2.column2).set_title("Figure: Boxplot repersenting outlier columnn
```

```
Out[74]: Text(0.5, 1.0, 'Figure: Boxplot repersenting outlier columns.')
```

Figure: Boxplot repersenting outlier columns.



```
In [75]: outlier_region=data2[data2.column2>10000000]
print(outlier_region['column1'].iloc[-1],'is outlier value')
```

TotalUS is outlier value

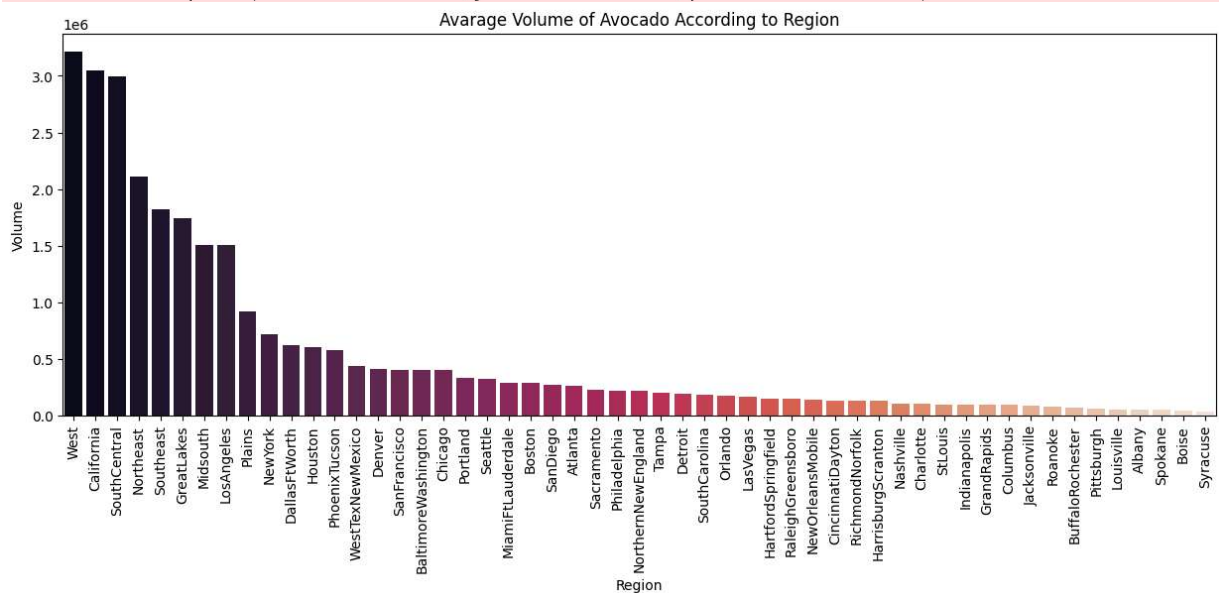
```
In [76]: outlier_region.index
data2 = data2.drop(outlier_region.index,axis=0)
```

```
In [77]: plot(data2,'Region','Volume')
```

C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12444\640296719.py:14: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax=sns.barplot(x=data.column1,y=data.column2,palette='rocket')
```

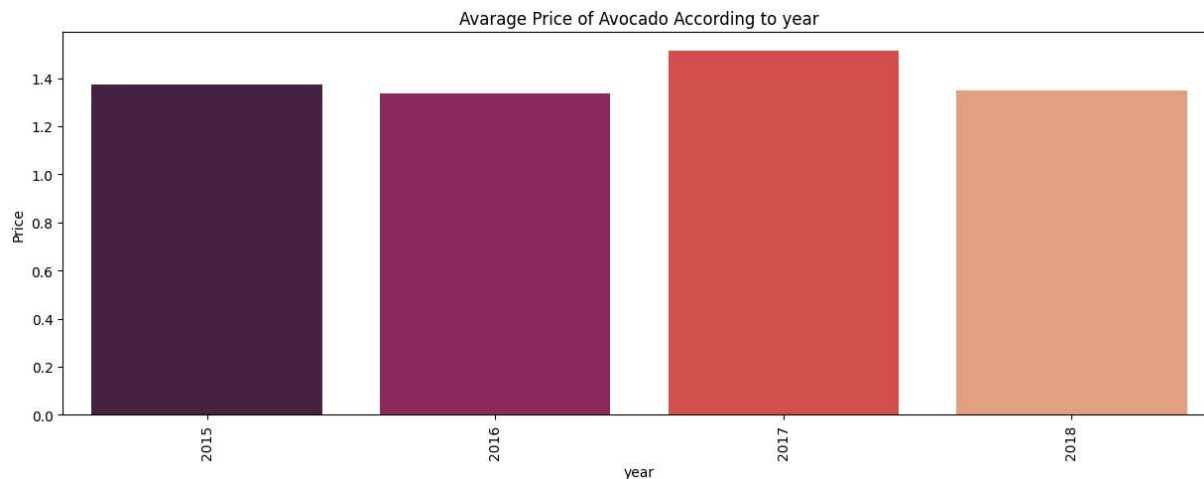


```
In [78]: data3 = get_avarge_between_two_columns(df,'year','AveragePrice')
plot(data3,'year','Price')
```

C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12444\640296719.py:14: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax=sns.barplot(x=data.column1,y=data.column2,palette='rocket')
```

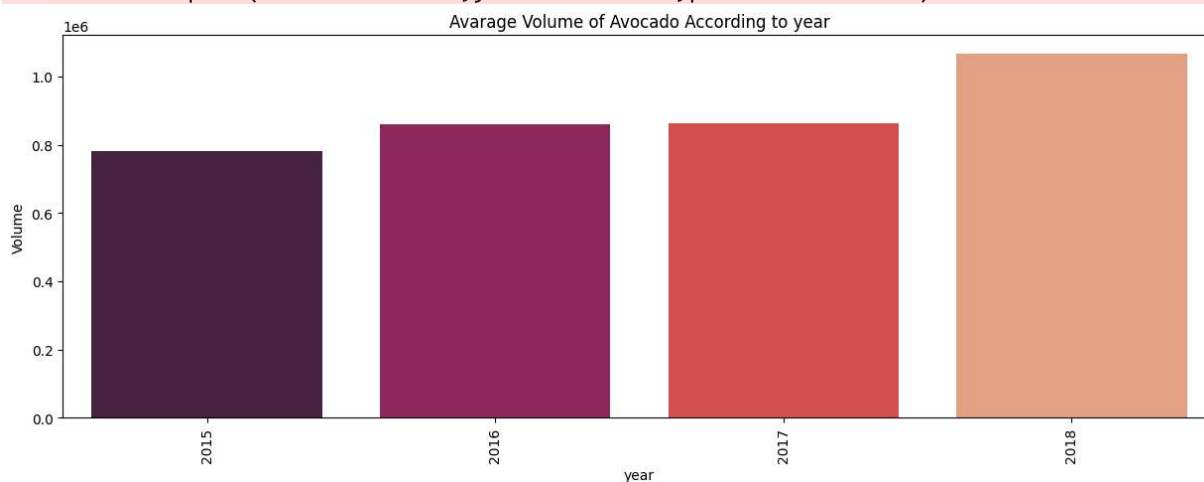


```
In [79]: data4 = get_avarge_between_two_columns(df, 'year', 'Total Volume')
plot(data4, 'year', 'Volume')
```

C:\Users\Lenovo\AppData\Local\Temp\ipykernel_12444\640296719.py:14: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax=sns.barplot(x=data.column1,y=data.column2,palette='rocket')
```



```
In [80]: df['region'] = df['region'].astype('category')
df['region'] = df['region'].cat.codes

df['type'] = df['type'].astype('category')
df['type'] = df['type'].cat.codes
```

```
In [81]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18249 entries, 0 to 18248
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   AveragePrice    18249 non-null  float64
1   Total Volume    18249 non-null  float64
2   Total Bags      18249 non-null  float64
3   Small Bags      18249 non-null  float64
4   Large Bags      18249 non-null  float64
5   XLarge Bags     18249 non-null  float64
6   type            18249 non-null  int8
7   year            18249 non-null  int64
8   region          18249 non-null  int8
dtypes: float64(6), int64(1), int8(2)
memory usage: 1.0 MB
```

```
In [82]: df.head()
```

Out[82]:

	AveragePrice	Total Volume	Total Bags	Small Bags	Large Bags	XLarge Bags	type	year	region
0	1.33	64236.62	8696.87	8603.62	93.25	0.0	0	2015	0
1	1.35	54876.98	9505.56	9408.07	97.49	0.0	0	2015	0
2	0.93	118220.22	8145.35	8042.21	103.14	0.0	0	2015	0
3	1.08	78992.15	5811.16	5677.40	133.76	0.0	0	2015	0
4	1.28	51039.60	6183.95	5986.26	197.69	0.0	0	2015	0

```
In [83]: x = df.drop(['AveragePrice'],axis=1)
y = df['AveragePrice']

# split data into training and testing dataset
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=0.3,random_state=
```

```
In [84]: print('training set:', x_train.shape['_'],y_train.shape[0],'samples')
print('testing set:', x_test.shape['_'],y_test.shape[0],'samples')

training set: (12774, 8) _ 12774 samples
testing set: (5475, 8) _ 5475 samples
```

```
In [85]: model = LinearRegression()
model.fit(x_train,y_train)
```

Out[85]:

LinearRegression

LinearRegression()

```
In [89]: test_pred = model.predict(x_test)
test_score = r2_score(y_test,test_pred)
print("The accuracy of testing dataset ",test_score*100)
```

The accuracy of testing dataset 38.580741764370785

```
In [90]: train_pred = model.predict(x_train)
         train_score = r2_score(y_train,train_pred)
         print("The accuracy of training dataset ",train_score*100)
```

The accuracy of training dataset 39.706860424108356

In []: